DETECTRON2: Train & Evaluate

```
# Register Dataset

from detectron2.data.datasets import register_coco_instances

register_coco_instances("train_dataset", {}, "/path/train/coco_annotations.json",

"/path/train/images")

register_coco_instances("valid_dataset", {}, "/path/valid/coco_annotations.json",

"/path/valid/images")

register_coco_instances("test_dataset", {}, "/path/test/coco_annotations.json",

"/path/test/images")

# Train Model

python3 train_net.py \

--config-file configs/COCO-InstanceSegmentation/mask_rcnn_R_50_FPN_3x.yaml \

--num-gpus 1 \

SOLVER.IMS_PER_BATCH 4 \

SOLVER.BASE_LR 0.00025 \

SOLVER.MAX_ITER 30000 \

DATASETS.TRAIN "('train_dataset',)" \

DATASETS.TEST "('valid_dataset',)" \

OUTPUT_DIR ./output_fire_smoke

# Evaluate

python3 train_net.py \--config-file configs/COCO-nstanceSegmentation/mask_rcnn_R_50_FPN_3x.yaml

--eval-only \

MODEL.WEIGHTS output_fire_smoke/model_final.pth \

DATASETS.TEST "('test_dataset',)"
```

YOLOv7: Train & Predict

```
# Clone & Setup

git clone https://github.com/WongKinYiu/yolov7

cd yolov7 && git checkout 44f30af0daccb1a3baecc5d80eae22948516c579

cd seg

pip install -r requirements.txt

wget https://github.com/WongKinYiu/yolov7/releases/download/v0.1/yolov7-seg.pt

# Train YOLOv7

python segment/train.py \

--batch-size 16 \

--img-size 640 \

--epochs 300 \

--data /path/to/dataset/data.yaml \

--weights yolov7-seg.pt \

--device 0 \

--name fire_smoke_seg

# Predict

python segment/predict.py \

--weights runs/train-seg/fire_smoke_seg/weights/best.pt \

--conf 0.25 \

--source /path/to/dataset/test/images \

--name fire_smoke_results
```

YOLOv9 (Ultralytics): Validation

```
yolo task=segment mode=val \model="/path/to/yolov9/weights/best.pt" \

data="/path/to/data.yaml" \

imgsz=640 batch=1 \

conf=0.25 save=True save_txt=True save_conf=True save_json=True \

project=runs/val name=fire_detection_eval
```

U-Net (Keras): Train

```
model = unet_model(input_size=(256, 256, 3), num_classes=NUM_CLASSES)

model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

history = model.fit(train_images, train_masks, validation_data=(val_images, val_masks),

epochs=300, batch_size=16)
```

TFLite Inference with Masks and Boxes

```
interpreter = tf.lite.Interpreter(model_path='your_model.tflite')

interpreter.allocate_tensors()

input_details = interpreter.get_input_details()

output_details = interpreter.get_output_details()

image = cv2.imread(image_path)

image = cv2.resize(image, (640, 640)).astype(np.float32) / 255.0

image = np.expand_dims(image, axis=0)

interpreter.set_tensor(input_details[0]['index'], image)

interpreter.invoke()

boxes = interpreter.get_tensor(output_details[0]['index'])

masks = interpreter.get_tensor(output_details[1]['index'])
```